

Lecture 3: Decision Trees

Alexis BOGROFF

Course on Classification Problems @Albert School

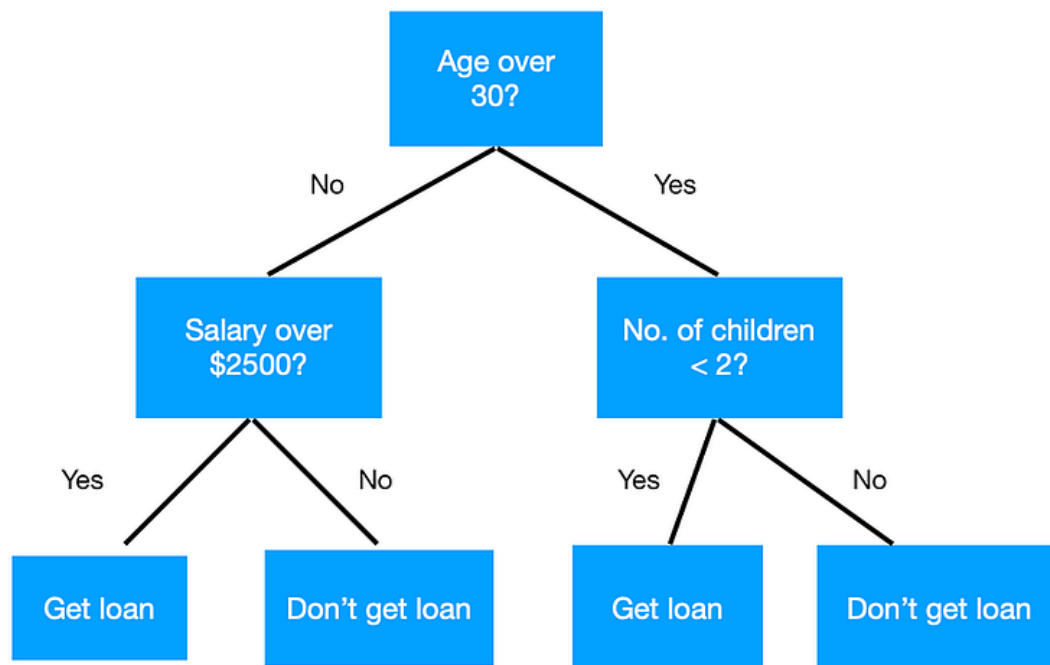
January 28, 2026

Introduction to Decision Trees

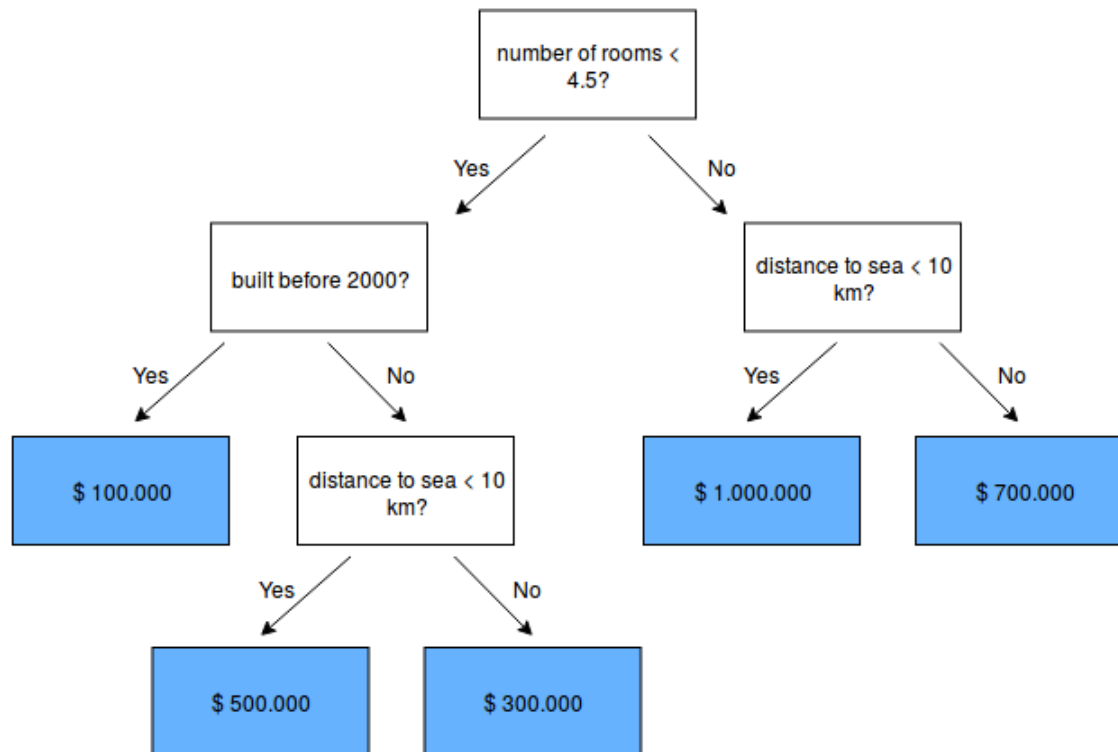
CART Algorithm

What is a Decision Tree?

- A supervised learning algorithm used for classification and regression tasks.
- Mimics human decision-making (yes/no).
- Represents data in a tree-like structure.
- Splits are computed recursively.



- **Classification Trees:** Output is a category.
- **Regression Trees:** Output is a continuous value.



To speak the language of Decision Trees, we must name the parts:

- **Root Node:** The top node representing the entire population. It gets divided into two or more sets.
- **Internal Node** (or Decision Node): A sub-node that splits into further sub-nodes. It represents a decision rule (e.g., $Age > 30$).
- **Branch:** A subsection of the entire tree.
- **Leaf Node** (or Terminal Node): Nodes that do not split. They contain the final output (Class prediction or Regression value).

- The CART algorithm underlies modern decision tree implementations.
- It uses the Gini Index to measure impurity in a dataset and performs binary splits.
- For classification, given a dataset D with c classes:
 - p_i is the probability of an instance belonging to class i :

$$p_i = \frac{|D_i|}{|D|}$$

$$Gini(D) = 1 - \sum_{i=1}^c p_i^2$$

- A lower Gini Index indicates a purer node, meaning it predominantly contains instances of a single class.

- For a general computation of Gini Index, when a dataset is split into k child nodes D_j , we compute the weighted sum of the Gini Indices:

$$Gini_{split} = \sum_{j=1}^k \frac{|D_j|}{|D|} Gini(D_j)$$

- Since CART implements a binary split there are only two partitions at each split, i.e. $k = 2$, so the formula simplifies to:

$$Gini_{split} = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2)$$

- With $|D_j|$ the number of instances in subset D_j .
- The optimal split is the one that results in the lowest $Gini_{split}$.

- While CART uses Gini, other algorithms (ID3, C4.5) use **Entropy**.
- **Entropy** measures the disorder or uncertainty in the data.

$$H(D) = - \sum_{i=1}^c p_i \log_2(p_i)$$

- **Information Gain (IG)**: We choose the split that reduces Entropy the most.

$$IG(D, \text{split}) = H(D) - \sum_{j=1}^k \frac{|D_j|}{|D|} H(D_j)$$

- **Comparison:**
 - Gini is computationally faster (no log to compute).
 - Entropy tends to produce slightly more balanced trees.
 - In practice: Results are often very similar (95% agreement).

Example: Computing Gini Index

- Consider a dataset with 10 instances and two classes: A and B.
- Before splitting:

$$\begin{aligned} \text{Gini}(D) &= 1 - (p_A^2 + p_B^2) \\ &= 1 - \left(\left(\frac{5}{10} \right)^2 + \left(\frac{5}{10} \right)^2 \right) \\ &= 1 - (0.5^2 + 0.5^2) \\ &= 1 - (0.25 + 0.25) \\ &= 0.5 \end{aligned}$$

Feature X	Class
5.1	A
4.9	A
5.5	A
6.1	B
5.8	B
6.2	B
5.7	B
5.9	A
6.3	B
6.5	A

Step 1: Sort Unique Values of the Feature

- Given feature values: [5.1, 4.9, 5.5, 6.1, 5.8, 6.2, 5.7, 5.9, 6.3, 6.5]
- Sort in ascending order: [4.9, 5.1, 5.5, 5.7, 5.8, 5.9, 6.1, 6.2, 6.3, 6.5]

Step 2: Compute Midpoints Between Consecutive Values

- Possible split points:

$$\frac{4.9 + 5.1}{2}, \frac{5.1 + 5.5}{2}, \frac{5.5 + 5.7}{2}, \dots$$
$$= 5.0, 5.3, 5.6, 5.75, 5.85, 6.0, 6.15, 6.25, 6.4$$

Step 3: Evaluate Gini Index for Each Split

- Split dataset at each midpoint and compute $Gini_{split}$.
- The split that minimizes impurity is chosen.

Example: Computing Gini a Poor Gini Split

- Let's consider a split at 5.85

Left node ($X \leq 5.85$):

- 3 instances of A, 2 of B.
- $Gini(Left) = 1 - (0.6^2 + 0.4^2)$
- $= 0.48$

Right node ($X > 5.8$):

- 2 instances of A, 3 of B.
- $Gini(Right) = 1 - (0.4^2 + 0.6^2)$
- $= 0.48$

Compute weighted Gini Split:

$$Gini_{split} = (0.5 \times 0.48) + (0.5 \times 0.48) = 0.48$$

- This split does not significantly reduce impurity.

Example: Computing the Best Gini Split

- Let's consider a split at $X = 5.6$

Left node ($X \leq 5.6$):

- 3 instances of A, 0 of B.
- $Gini(Left) = 0.0$ (pure node)

Right node ($X > 5.5$):

- 2 instances of A, 5 of B.
- $Gini(Right) = 1 - (0.285^2 + 0.714^2)$
- $Gini(Right) = 0.408$

Compute weighted Gini Split:

$$Gini_{split} = (0.3 \times 0) + (0.7 \times 0.408) = 0.286$$

- This split significantly reduces impurity, leading to better classification.

- How does the tree decide the perfect structure?
- **Recursive Partitioning:** It builds the tree step-by-step, splitting data into smaller subsets recursively.
- **Greedy Algorithm:**
 - At each step, the algorithm makes the locally optimal choice (best split *now*).
 - It does **not** look ahead to see if a slightly worse split now would lead to a much better split later.
 - Consequence: Decision Trees might not find the globally optimal tree, but they are fast and effective.

- Unlike classification trees, regression trees predict continuous values.
- Instead of Gini impurity, they minimize Mean Squared Error (MSE) at each split:

$$MSE(t) = \frac{1}{|D_t|} \sum_{i \in D_t} (y_i - \bar{y}_t)^2$$

- Where:
 - y_i is the actual value of instance i .
 - $\bar{y}_t$ is the mean value of instances in node t .
- The best split minimizes the weighted sum of the MSE in child nodes, thereby reducing prediction error.

- Feature importance is thus based on MSE reduction instead of impurity reduction.
- This method is known as **Mean Decrease in Impurity (MDI)**. The score is computed as:

$$I(f) = \sum_t \Delta MSE(t) \times P(t)$$

- Where:
 - $\Delta MSE(t)$ is the reduction in Mean Squared Error at node t .
 - $P(t)$ is the probability of reaching node t .

Example: Splitting in a Regression Tree

- Consider a dataset predicting Parisian house prices based on square meters (m^2).
- Let's split at 55m^2

Size (m^2)	Price (EUR)
45	600,000
50	650,000
55	700,000
65	800,000
70	850,000

- Mean Squared Error (MSE) before the split:
 $8,600,000,000 \text{ EUR}^2$
- MSE after the split:
 $500,000,000 \text{ EUR}^2$

Left Node (≤ 55):

- 45m^2 : 600,000 EUR
- 50m^2 : 650,000 EUR
- Mean Prediction: 625,000 EUR

Right Node (> 55):

- 65m^2 : 800,000 EUR
- 70m^2 : 850,000 EUR
- Mean Prediction: 825,000 EUR

- Decision trees can be pruned to prevent overfitting.
- Two main types of pruning exist:

Pre-Pruning (Early Stopping)

- Stops tree growth early before reaching full depth.
- Limits complexity by using hyperparameters like:
 - `max_depth`: Maximum depth of the tree.
 - `min_samples_split`: Minimum samples needed to split a node.
 - `min_samples_leaf`: Minimum samples required at a leaf.
 - `max_leaf_nodes`: Limits the number of leaf nodes.
- Risk: Can lead to underfitting if stopped too early.

Post-Pruning

- CCP is a post-pruning technique applied after the tree is fully grown.
- It removes unnecessary branches to reduce complexity.
- The cost complexity function is:

$$R_{\alpha}(T) = R(T) + \alpha|T|$$

where:

- $R(T)$ is the impurity (e.g., Gini, MSE).
- $|T|$ is the number of leaf nodes.
- α is the complexity parameter controlling pruning strength.

- CCP is not applied by default ('ccp_alpha=0.0').
- Steps to apply CCP:
 1. Train a full decision tree.
 2. Train trees for different α values and choose the best one ¹

Example Code:

```
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier(ccp_alpha=0.01)
clf.fit(X_train, y_train)
```

¹To go further with a clever selection of α values, see https://scikit-learn.org/stable/auto_examples/tree/plot_cost_complexity_pruning.html

- Importance scores are assigned to features based on how much they reduce impurity.
- It corresponds to how often the feature was used and how strong was the impurity reduction each time the feature was used.
- The importance score for a feature f is computed as:

$$I(f) = \sum_{t \in \text{nodes where } f \text{ is used}} \Delta Gini(t) \times P(t)$$

Where:

- $\Delta Gini(t)$ is the impurity reduction at node t :

$$\Delta Gini(t) = Gini(t) - \left(\frac{|D_L|}{|D|} Gini(D_L) + \frac{|D_R|}{|D|} Gini(D_R) \right)$$

- $P(t)$ is the probability of reaching node t :

$$P(t) = \frac{|D_t|}{|D_{\text{total}}|}$$

- Feature importance helps in feature selection and model interpretability:
- Higher score $I(f) \rightarrow$ Feature is used frequently and significantly reduces impurity
 $\rightarrow$ More important.
- Lower score $I(f) \rightarrow$ Feature contributes little to reducing impurity
 $\rightarrow$ Less important.

Extracting Feature Importance in Scikit-Learn:

```
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)
importance_scores = clf.feature_importances_
print(importance_scores)
```

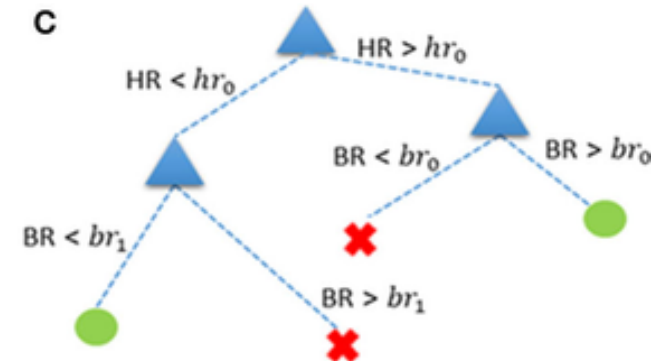
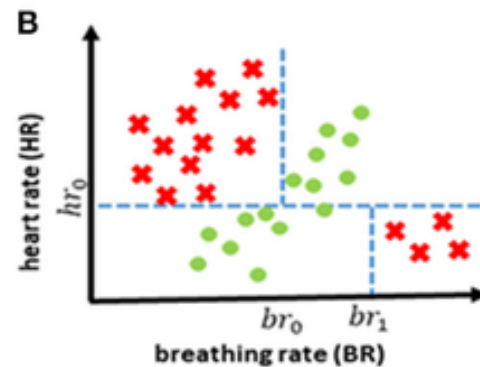
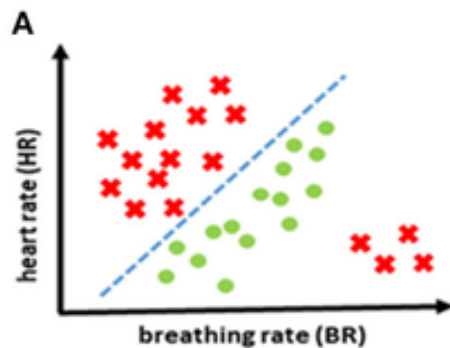
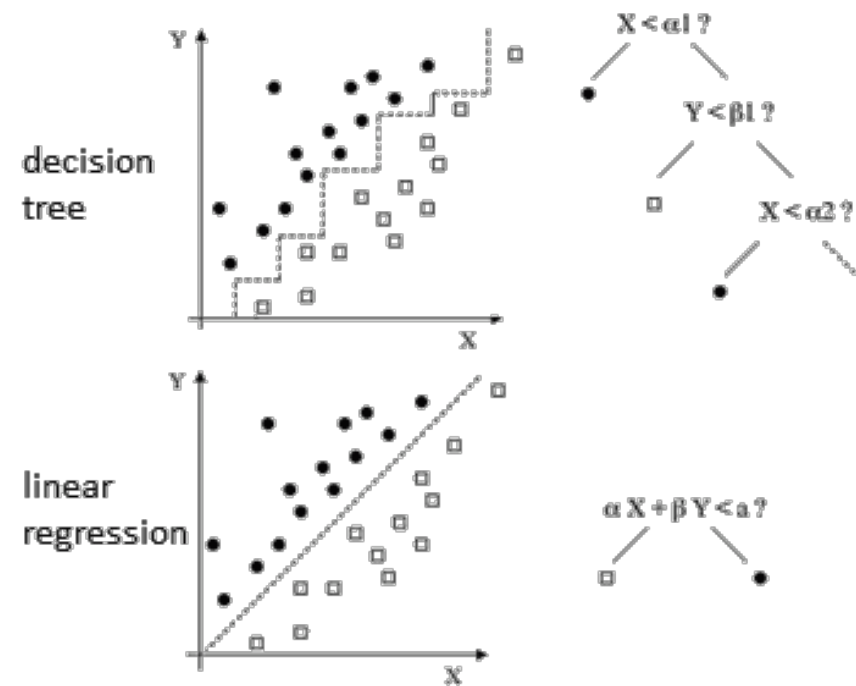
- Classification
 - Use Confusion Matrix to analyze predictions.
 - Compute accuracy, precision, recall, and F1-score.
- Regression
 - Use Mean Squared Error (MSE) to measure error.
 - Use R^2 Score to assess model goodness-of-fit.

Key Limitations:

- Overfitting: Decision trees tend to overfit if not pruned or regularized properly.
- Instability: Small changes in data can lead to a completely different tree structure.
- Lack of Extrapolation: Decision trees cannot extrapolate beyond the training data range.
 - Example: A regression tree trained on house prices between 30m² and 110m² cannot predict for 150m² accurately.
- Biased towards dominant classes: If class imbalance exists, trees may favor the majority class.
- High variance: Sensitive to the training set, requiring ensemble methods (Random Forests, Boosting) for stability.

Visualizing the Decision Surface

- **Orthogonal Boundaries:**
 - Logistic Regression draws diagonal lines (linear combinations of features)
 - Decision Trees draw boundaries **perpendicular to the feature axes**.
- Creates "box-like" / "step-wise" decision regions.
- **Implication:** To approximate a diagonal boundary, a Decision Tree needs many splits (staircase effect), which increases complexity.



Exercise 1: Understanding Decision Tree Splits

Task: Given the following dataset, determine the best split using Gini Index.

Feature X	Class
2.5	A
3.0	A
3.5	B
4.0	B
4.5	A
5.0	B

Questions:

- Compute the Gini Index before splitting.
- Evaluate possible splits and compute Gini Split.
- Determine the best split.

Exercise 2: Constructing a Regression Tree

Task: Build a regression tree using the following data (predict house prices in Paris).

Size (m ²)	Price (EUR)
30	400,000
50	600,000
70	800,000
90	1,000,000
110	1,200,000

Questions:

- Compute the MSE before splitting.
- Find the optimal split by minimizing MSE.
- Predict prices for new data points using your tree.

Exercise 3: Decision Tree Hyperparameters

Task: Investigate the impact of hyperparameters on decision trees.

- Train a Decision Tree Classifier on a dataset.
- Display the tree structure using `sklearn's plot_tree` method.
- Experiment with different values for:
 - `max_depth`
 - `min_samples_split`
 - `min_samples_leaf`
- Observe how tree structure and accuracy change.

Questions:

- How does increasing `max_depth` affect overfitting?
- What happens when `min_samples_leaf` is too high?
- How do you determine the best hyperparameters?